ASSIGNMENT REPORT 1: PROCESS AND THREAD IMPLEMENTATION

OS 2034, OPERAATING SYSTEMS

Ali Şişik
alisisik@posta.mu.edu.tr

Monday 4th May, 2026

Abstract

Overall, this report looks at the performance and time differences between multithreading and multiprocessing in Python. The main goal is to find the ”Efficiency Crossover Point.” This is the exact moment when the extra workload of running separate processes actually becomes faster than dealing with the Global Interpreter Lock (GIL) limits in threading. By testing simulated I/O-bound and CPU-bound tasks, this study shows how shared memory, separate address spaces, and communication between processes affect the overall run time.

1 Introduction

Python’s GIL prevents threads from running code at the exact same time. This lab explores how the GIL blocks true parallelism in CPU-heavy tasks. It also compares threads (which share memory) with processes (which use separate memory). Finally, the experiment measures the ”Communication Tax”—the extra time it takes to share data between completely separate processes.

2 Technical Discussion

To understand the results of this experiment, three key technical concepts must be considered:

2.1 The Global Interpreter Lock (GIL)

The GIL is a mutex in Python that allows only one thread to execute at a time. While it makes memory management easier, it prevents true parallelism in CPU-heavy tasks. This is why threading often fails to improve performance for math-intensive operations.

2.2 Address Spaces

Threads share the same memory space, which makes them ”lightweight” and fast to start. However, processes have isolated address spaces. This isolation allows processes to bypass the GIL and run on multiple CPU cores simultaneously, but it makes them ”heavier” to create.

2.3 The Communication Tax

Since processes do not share memory, they must serialize (pickle) data to send it to each other. This extra work, along with the time the operating system takes to start a new process, creates a "Communication Tax." This is the roughly 1-second delay we see in our multiprocessing results.

3 Assignments

In this project, I wrote a Python script to test two different scenarios. The goal was to see how threads and processes handle different types of tasks.

3.1 Scenario 1: The I/O Simulation

For the first assignment, I wanted to test an I/O-bound task. I created a script that runs 50 tasks, where each task just waits for 0.1 seconds using `time.sleep(0.1)`.

I tested this in three ways:

- Sequential execution (one by one)
- Multi-threading (using 5 threads)
- Multi-processing (using 5 processes)

I also used a shared counter to track the completed tasks. To prevent race conditions, I used `threading.Lock` and `multiprocessing.Lock`.

3.2 Scenario 2: Finding the Break-even Point

For the second assignment, I tested a CPU-heavy task. I wrote a function that calculates the sum of squares for a large list of numbers.

To find the "break-even" point, I gradually increased the workload (the N value) from 10,000 up to 200,000,000. I split the work into two parts and compared how long it took for threads and processes to finish the math. The goal was to find the exact data size where multiprocessing finally becomes faster than threading.

4 Results

The performance tests were conducted on a system equipped with a Intel Core i7 13th gen processor and 16 GB of RAM.

4.1 Scenario 1: I/O-Bound Task Performance

In the first scenario, 50 I/O bound tasks (`time.sleep(0.1)`) were executed. The results clearly show that threading is the most efficient method for I/O tasks:

- **Sequential Execution:** 5.0218 seconds
- **Multi-threading (5 Threads):** 1.0078 seconds
- **Multi-processing (5 Processes):** 1.9039 seconds

4.2 Scenario 2: CPU-Bound Task and Break-even Point

In the second scenario, the workload (N) was increased to find the efficiency cross-over point. The results are summarized in Table 1.

Workload (N)	**Thread Time (s)**	**Process Time (s)**	**Winner**
10,000	0.0016	1.0235	Threading
100,000	0.0048	1.0146	Threading
1,000,000	0.0441	1.0246	Threading
10,000,000	0.4108	1.2099	Threading
50,000,000	2.0370	2.1746	Threading
100,000,000	4.0176	3.2150	**Multiprocessing**
200,000,000	8.5431	6.0731	**Multiprocessing**

Table 1: CPU-bound task results showing the break-even point.

5 Conclusion

This lab demonstrated that threading is superior for I/O-bound tasks because it avoids the heavy overhead of creating new processes while the GIL is released during wait times. However, for CPU-bound tasks, the GIL becomes a bottleneck.

Our data shows a "Communication Tax" of about 1 second for multiprocessing on this hardware. The efficiency cross-over point was found between $N = 50,000,000$ and $N = 100,000,000$. Beyond this point, the benefits of true parallelism in multiprocessing outweigh the initial cost of process creation.